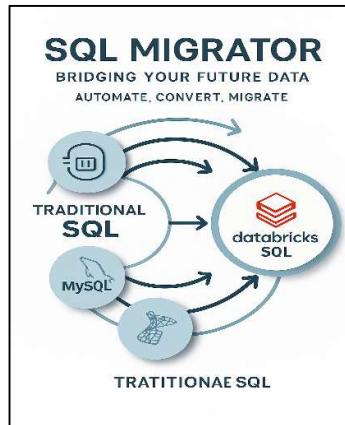


SQL Migrator

SQL Migrator is an AI-assisted, rules-driven SQL dialect conversion tool designed to address real-world limitations observed in existing SQL conversion solutions, particularly the Databricks Assistant. The project focuses on reliability, performance, and enterprise-scale SQL migration from T-SQL to Databricks SQL.



Objective

The primary objective of SQL Migrator is to provide a **clear, measurable, and practical comparison** between:

- Databricks Assistant
- SQL Migrator (this project)

The comparison is evaluated across three core dimensions:

1. Accuracy
2. Performance
3. Flexibility

Limitations of Databricks Assistant

While Databricks Assistant is a powerful AI-driven tool, several limitations were identified during real-world SQL migration efforts:

1. Requires precise and time-consuming prompt engineering for each query type such as INSERT, UPDATE, JOIN, and ALTER.
2. Many Microsoft SQL Server features are unsupported, including:
 - System stored procedures
 - Variable declarations
 - Transactions
 - TRY...CATCH blocks
 - Metadata and system functions
3. Large SQL scripts (500+ lines) often take 5–10 minutes or more to process.
4. Multiple iterations are frequently required to obtain correct output.

5. Prompts lack reusability and clear responsibility separation.
6. Rigid prompt structures reduce adaptability across different SQL patterns.
7. Available only to premium-tier users, limiting accessibility for startups and smaller teams.
8. Direct migration of SQL scripts (for example, from SQL Server or MySQL) is unreliable and often results in compatibility issues.

Key Advantages of SQL Migrator

SQL Migrator was designed specifically to overcome the above challenges.

1. Accuracy

- **Databricks Assistant**
 - Uses general-purpose LLMs.
 - Works well for simple to moderately complex queries.
 - Struggles with deeply nested logic and less-common SQL features.
- **SQL Migrator**
 - Uses deterministic, rules-based logic combined with AI summarization.
 - Handles both standard and complex SQL queries reliably.
 - Preserves semantic correctness and execution intent.

2. Performance

- **Databricks Assistant**
 - Response time depends on model load and prompt complexity.
 - Large scripts can take 5–10 minutes or more.
 - Often requires re-execution to fix partial or incorrect output.
- **SQL Migrator**
 - Lightweight and optimized for batch processing.
 - Handles large scripts efficiently using chunking and parallel execution.
 - Typical processing time for large scripts is 1–2 minutes.

3. Flexibility

- **Databricks Assistant**
 - Requires carefully crafted prompts per query type.
 - Prompt complexity increases rapidly with SQL variety.
 - Reduced adaptability across use cases.
- **SQL Migrator**
 - No prompt engineering required from the user.
 - Upload-and-convert workflow.
 - Converts T-SQL to Databricks SQL with approximately 90 percent accuracy out of the box.

Solution Overview

SQL Migrator implements a **hybrid conversion strategy** combining deterministic transformation rules with AI-powered analysis.

What Was Built

A custom SQL conversion engine that:

- Automatically transforms T-SQL into Databricks SQL.
- Handles real-world enterprise SQL scripts.
- Produces clean, executable output formats.

Key Capabilities

- Fast, reliable SQL conversion.
- Supports a wide range of SQL operations:
 - INSERT
 - UPDATE
 - JOIN
 - ALTER
 - DDL and DML statements
- Actively mitigates known Databricks Assistant limitations.
- Chunk-based processing for scalability.
- Parallel execution for performance.

Business Impact

SQL Migrator enables:

- Faster cloud data platform modernization.
- Reduced manual SQL rewriting effort.
- Reliable migration pipelines for enterprise data warehouses.
- Lower dependency on expensive premium AI tooling.

It is particularly well-suited for organizations migrating legacy SQL Server workloads to Databricks-based Lakehouse architectures.

Positioning Summary

Aspect	Databricks Assistant	SQL Migrator
Accuracy	Medium–High	High
Performance	Slow for large files	Optimized
Flexibility	Prompt-dependent	Upload-and-convert
Cost Access	Premium-only	Open and extensible
Enterprise Readiness	Limited	High

If you want, I can next:

- Merge this seamlessly into your earlier README version
- Create a dedicated ABOUT.md
- Convert this into a website-friendly Markdown section
- Produce architecture or comparison diagrams
- Write a one-page executive summary for stakeholders

Feedback

1. It should be able to convert SCD type 1 and type 2 stored procedures into Spark SQL.
2. UI Enhancement:
 - a. Place the convert button from to up or middle.
 - b. Put the clear button on both cards.